图像灰度世界白平衡测试

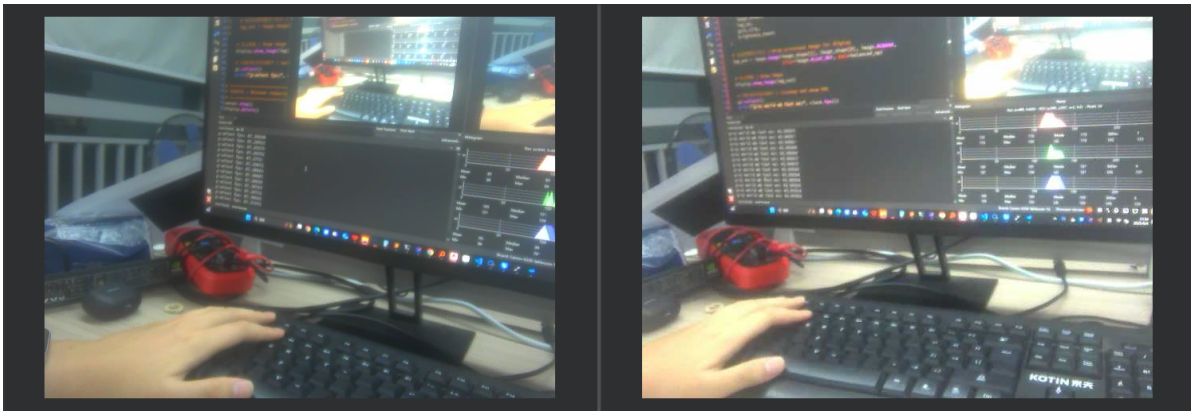
图像灰度世界白平衡测试
- 例程效果
- 原理讲解
  - 什么是图像灰度世界白平衡
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置帧率计时器
    - 设置白平衡参数
    - 图像处理与白平衡操作
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/28.rgb888_white_balance_gray_world_fast_ex.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像灰度世界白平衡处理，通过可调参数优化图像颜色校正和亮度增强。

> 左侧为原始图像，右侧为处理后的图像



原理讲解

什么是图像灰度世界白平衡

图像灰度世界白平衡是一种简单的颜色校正技术，基于"灰度世界假设"，即在自然场景中，图像的平均颜色趋于中性灰。它通过调整图像的RGB通道，使每个通道的平均值相等，从而消除由于光源导致的颜色偏移。该算法在嵌入式设备上特别有用，因为它计算量小且高效。

- **操作原理**：灰度世界白平衡首先计算图像每个颜色通道（R、G、B）的平均值，然后计算每个通道的增益，使其平均值接近全局平均值（通常设为中性灰）。可选地，可以应用亮度增强以改善整体亮度。公式简化版为：对于每个通道，增益 = (全局平均值 / 通道平均值)，然后乘以像素值。最后，应用增益限制以防止过度饱和。

- **效果**：该操作可以有效修正图像的色温问题，使颜色更自然逼真，同时通过参数调整控制增益和亮度，避免颜色失真或过度曝光。处理后的图像更适合显示或进一步分析。
- **应用场景**：灰度世界白平衡广泛用于相机图像处理、视频实时校正、手机摄影和机器人视觉等领域。在嵌入式系统中，它是快速白平衡的首选，因为它不需要复杂的校准硬件，仅依赖算法。

在实际应用中，可调参数如增益限制和亮度增强可以根据环境光线和图像特性进行优化，以平衡颜色准确性和视觉效果。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import _thread
import cv_lite                # cv_lite 图像处理扩展模块 / cv_lite C extension
module
import ulab.numpy as np       # ulab 数组模块 / NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960,fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合进行颜色校正处理）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置帧率计时器

```
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

设置白平衡参数

```
gain_clip = 2.5              # 增益限制系数，防止偏色 / Gain limit to prevent color
blowout
brightness_boost = 1.25      # 亮度增强系数 / Global brightness boost
```

- `gain_clip`：增益限制系数，用于防止颜色通道过度放大导致的颜色失真。
- `brightness_boost`：亮度增强系数，用于整体提升图像亮度。

图像处理与白平衡操作

```
while True:
    clock.tick()  # 记录当前帧开始时间 / Start timing

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray
reference (HWC)

    # 使用 cv_lite 进行加速灰度世界白平衡处理（可调参数）
    # Apply fast gray world white balance with tunable parameters
    balanced_np = cv_lite.rgb888_white_balance_gray_world_fast_ex(
        image_shape,
        img_np,
        gain_clip,
        brightness_boost
    )

    # 包装图像用于显示 / Wrap processed image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                          alloc=image.ALLOC_REF, data=balanced_np)

    # 显示图像 / Show image
    Display.show_image(img_out)

    # 回收内存并打印帧率 / Cleanup and show FPS
    gc.collect()
    print("gray world wb fast ex:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。

- **白平衡处理**：调用 `cv_lite.rgb888_white_balance_gray_world_fast_ex()` 函数执行灰度世界白平衡，返回处理后的图像数组。
- **图像包装和显示**：将处理结果包装为 RGB888 图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `gain_clip`：增益限制系数，值越大（如 1.5-3.0）允许更大范围的颜色调整，但可能导致饱和；值越小（如 1.0）保持原色，建议从 2.0 开始测试，以平衡颜色校正和防止偏色。
- `brightness_boost`：亮度增强系数，值大于 1.0（如 1.0-1.5）增强整体亮度，改善暗部细节；值等于 1.0 无增强，值过大（如 2.0）可能导致过曝，建议根据环境光线调整，从 1.1 开始测试。